

Rooting the Teclast P30

Magisk root and debloat for Allwinner A523 / T527 tablets, using FEL mode and a bare-metal eMMC driver. Standard fastboot flashing does not work on this hardware.

Verified on	Teclast P30 (P30_ROW), Android 14, build UP1A.231105.001.A1
SoC	Allwinner A523 / T527 — sun55iw3p1, board “saturn”, arm64-v8a
Storage	58 GB eMMC, A/B slots, separate init_boot partition (GKI)
Host	Linux (Ubuntu 26.04 used here), USB-A to USB-C cable
Result	Magisk 30.7, uid=0 root, 41 packages removed, boots clean

DESTRUCTIVE

Unlocking the bootloader erases everything on the tablet. There is no way around this — it is the standard Android anti-theft behaviour, and it happens the moment you run the unlock command. Back up anything you care about before you start. This guide assumes a device you are willing to wipe.

NOTE

Nothing here can permanently brick the tablet. The Allwinner boot ROM (FEL mode) lives in mask ROM and cannot be overwritten, so the device is always reachable over USB even when it will not boot. That is your safety net, and it is why the backup step matters more than any other.

BEFORE YOU START

What you need

Everything below is installed on the **host Linux machine**, not the tablet.

Distribution packages

On Debian / Ubuntu and derivatives:

```
sudo apt-get update
sudo apt-get install -y \
    android-sdk-platform-tools-common \
    adb fastboot \
    build-essential \
    gcc-arm-none-eabi \
    libusb-1.0-0-dev \
    pkg-config \
    git curl unzip \
    usbutils
```

Package	Why it is needed
adb, fastboot	Talk to the tablet in Android and bootloader modes
build-essential	Compile xfel on the host (gcc, make)
gcc-arm-none-eabi	Cross-compile the bare-metal ARM eMMC driver
libusb-1.0-0-dev	xfel links against libusb to reach FEL over USB
pkg-config	Locates libusb during the xfel build
usbutils	Provides lsusb, used constantly to confirm device mode
git, curl, unzip	Fetch the two repositories and the Magisk APK

WATCH OUT

Do **not** rely on the distro sunxi-tools package. Its sunxi-fel binary crashes on the A523 with an assertion failure (aw_read_usb_response: Assertion `strcmp(buf, "AWUS") == 0' failed). The A523 needs xfel, built from source. Installing sunxi-tools is harmless but it will not do the job.

Source repositories

```
# FEL mode tool with first-class A523/T527 support
git clone --depth 1 https://github.com/xboot/xfel.git
cd xfel && make && cd ..
# -> produces ./xfel/xfel

# Bare-metal eMMC driver that runs from SRAM via FEL
git clone --depth 1 https://github.com/chrislennon/A523-root.git
cd A523-root/tools && make fel-emmc.bin && cd ../..
# -> produces fel-emmc.bin (~5.8 KB)
```

Add xfel to your PATH for the rest of the procedure — the eMMC helper script expects to find it there:

```
export PATH="$PWD/xfel:$PATH"
```

USB permissions

FEL mode presents as a raw USB device that your user cannot open by default. Install a udev rule once:

```
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="1f3a", ATTR{idProduct}=="efe8", MODE="0666"' \
| sudo tee /etc/udev/rules.d/71-sunxi-fel.rules
sudo udevadm control --reload-rules
sudo udevadm trigger
```

Knowing which mode the tablet is in

These USB IDs are the fastest way to tell what state the device is in. Keep `lsusb` handy throughout:

lsusb shows	Mode	Talk to it with
1f3a:4ee7	Android, booted normally	adb
1f3a:4ee1	Android, booted (alt. config)	adb
1f3a:efe8	FEL / flashing mode	xfel
Android Bootloader Interface	Fastboot	fastboot (limited)

STEP 1

Unlock the bootloader

ERASES ALL DATA

~5 MIN

On the tablet: **Settings** → **About tablet**, tap **Build number** seven times. Then **Settings** → **System** → **Developer options** and enable both **OEM unlocking** and **USB debugging**. Connect USB and accept the RSA fingerprint prompt.

```
adb devices -l
# 8622SFCEEB00004649    device    product:P30_ROW    model:P30
```

DESTRUCTIVE

The next command **wipes the tablet** — all apps, accounts, photos and settings. It is not reversible and there is no confirmation prompt on the host. Stop here if you have not backed up.

```
adb reboot bootloader
sleep 5
fastboot devices -l          # confirm it is there
fastboot oem unlock          # <-- WIPES THE DEVICE
fastboot reboot
```

The tablet reboots, erases userdata, and comes back at the first-run setup wizard. Walk through setup, then **re-enable Developer options and USB debugging** — the wipe cleared them, along with the ADB authorisation.

Confirm the unlock actually took

```
adb shell getprop ro.boot.flash.locked      # want: 0
adb shell getprop ro.boot.verifiedbootstate # want: orange
```

You should also now see an orange “*Your device has been unlocked and can't be trusted*” screen for about five seconds on every boot. That is expected and permanent while unlocked.

NOTE

Two Allwinner quirks worth knowing. `fastboot flashing unlock` (the modern spelling) fails with “*unknown cmd*” — you must use the older `fastboot oem unlock`. And `fastboot getvar all` returns “*secure mode, fastboot limited used*”; this bootloader implements only a tiny command subset, which is exactly why the rest of this guide uses FEL instead.

WATCH OUT

On some A523 devices `oem unlock` is a no-op and the real unlock flag has to be written into Allwinner's proprietary Secure Storage area by hand. On this P30 it worked properly. To check yours, dump Secure Storage in Step 4 and look for `fastboot_status_flag = "unlocked"`. If it is missing, see *If your bootloader will not unlock* near the end.

STEP 2

Enter FEL mode

PHYSICAL BUTTONS

FEL is the Allwinner boot ROM's USB recovery mode. It is the only reliable way to read and write this tablet's eMMC, and it is entered with a button sequence — not from software.

WATCH OUT

`adb reboot fel` looks like a shortcut and is not one. It drops the tablet into FEL for a fraction of a second and then continues booting, so any tool that tries to attach catches a half-open device and dies. Use the buttons.

The button sequence

- 1 Power the tablet **fully off** — hold Power, choose *Power off*. Not sleep.
- 2 Unplug the USB cable.
- 3 Press and **hold Volume Down**. Keep holding it for every step below.
- 4 Press **Power** for about 2 seconds, then release Power only.
- 5 Tap **Power** three more times, still holding Volume Down.
- 6 The screen must stay **completely black**. Any logo means it booted — start over.
- 7 Plug the USB cable back in.

Confirm you are in FEL. This is the single most important checkpoint in the guide:

```
lsusb | grep 1f3a
# Bus 001 Device 043: ID 1f3a:efe8 Allwinner Technology
# sunxi SoC OTG connector in FEL/flash mode

xfel version
# AWUSBFEX ID=0x00189000(A523/A527/T527/MR527) dflag=0x44 ...
```

NOTE

Unlike the `adb reboot fel` blip, a properly entered FEL mode **holds indefinitely** — the screen stays black and the device waits. You can take as long as you need between commands. It survives being moved; just do not unplug the cable.

TIMING

Everything from here until the final reset happens in FEL. Re-entering it means repeating the button sequence, so it is worth getting comfortable with it now.

STEP 3

Back up before touching anything

DO NOT SKIP

~10 MIN

This is your undo button. Without it, a bad write means reflashing stock firmware — and Teclast does not publish firmware for this model, so there may be nothing to reflash. Run these from A523-root/tools/ with the tablet in FEL.

First confirm the driver can actually talk to the eMMC:

```
cd A523-root/tools
./emmc-tool.sh info
# Sectors: 0x07480000 (122159104)   Size: 59648 MB (58 GB)
```

Read the partition table. This caches partition offsets, and the later dump and flash commands need it:

```
./emmc-tool.sh gpt
# 27 partitions: bootloader_a/b, boot_a/b, init_boot_a/b, super,
#                vbmeta_*, misc, userdata ...
```

Now take the backups. Keep these somewhere permanent — not /tmp:

```
mkdir -p ~/p30-backups

# The partition you are about to modify (8 MB, slowest of the three)
./emmc-tool.sh dump init_boot_a ~/p30-backups/init_boot_a_original.img

# Allwinner Secure Storage -- holds the bootloader unlock flag,
# plus your WiFi/BT MAC addresses and serial number
./emmc-tool.sh read 12288 256 ~/p30-backups/secure_storage_original.bin

# U-Boot environment
./emmc-tool.sh dump env_a ~/p30-backups/env_a_backup.bin

sha256sum ~/p30-backups/*
```

Check the backup is real

An 8 MB file of zeroes will not help you at 2am. Verify it is a genuine Android boot image before continuing:

```
xxd -l 16 ~/p30-backups/init_boot_a_original.img
# 00000000: 414e 4452 4f49 4421 ...  ANDROID!
```

The magic must read ANDROID!. For an Android 14 init_boot the kernel size is legitimately zero — the partition holds only the ramdisk — so that is not a sign of a bad dump.

NOTE

secure_storage_original.bin contains your tablet's WiFi and Bluetooth MAC addresses and its serial number in plain text. Keep it private; do not paste it into forums or issue trackers when asking for help.

Optional: check your unlock state

```
python3 unlock-bootloader.py dump ~/p30-backups/secure_storage_original.bin
# ...
# [14] fastboot_status_flag 8B plain CRC OK = "unlocked"
# Bootloader: UNLOCKED
```

STEP 4

Patch init_boot with Magisk

~5 MIN

Magisk does the patching **on the tablet**, so boot it out of FEL first:

```
xfel reset          # tablet reboots into Android
adb wait-for-device
```

Download Magisk from the official repository and verify it before installing. The signing certificate fingerprint below is F-Droid-style verification for Magisk — if it does not match, do not install:

```
curl -sLO https://github.com/topjohnwu/Magisk/releases/latest/download/Magisk-v30.7.apk
unzip -p Magisk-v30.7.apk META-INF/*.RSA \
| openssl pkcs7 -inform DER -print_certs \
| openssl x509 -noout -fingerprint -sha256

adb install Magisk-v30.7.apk
adb push ~/p30-backups/init_boot_a_original.img /sdcard/Download/
```

On the tablet, open **Magisk**. Accept *Additional Setup* if prompted, then:

- 1 Tap **Install**, next to “Magisk” at the top
- 2 Choose **Select and Patch a File**
- 3 Pick **Download → init_boot_a_original.img**
- 4 Tap **Let's Go** and wait for *All done!*

Pull the result back and sanity-check it:

```
adb pull /sdcard/Download/magisk_patched-30700_XXXXX.img .

# Must still be a boot image, must differ from the original,
# and must be the same size as the partition
xxd -l 16 magisk_patched-30700_XXXXX.img          # ANDROID!
cmp init_boot_a_original.img magisk_patched-*.img  # expect: differ
ls -l magisk_patched-*.img                        # expect: 8388608
```

NOTE

The patched ramdisk is often *smaller* than the original (here 2,151,998 → 1,986,870 bytes). That is normal — Magisk recompresses the ramdisk with its own settings. The file as a whole stays exactly 8,388,608 bytes because it is a full-partition image.

STEP 5

Flash the patched image

WRITES TO
eMMC

8-15 MIN

Put the tablet back into FEL mode (Step 2's button sequence), confirm with `xfel version`, then flash:

```
cd A523-root/tools
./emmc-tool.sh flash init_boot_a ~/magisk_patched-30700_XXXXX.img
# Chunk 1/256: sectors 599040-599103 (32KB)... OK
# ...
# Chunk 256/256: sectors 615360-615423 (32KB)... OK
# Done: wrote 16384 sectors to eMMC starting at sector 599040
```

TIMING

Expect this to take eight to fifteen minutes for 8 MB, and do not interrupt it. That is not a bug and your setup is not broken. A normal fastboot flash finishes in seconds; this does not, for three compounding reasons.

Why it is slow	Detail
1-bit bus	The driver defaults to 12 MHz on a 1-bit data bus. Normal flashing uses the full 8-bit bus at 52 MHz+ — roughly eight times the lane throughput before any other overhead.
PIO, not DMA	There is no DMA. The CPU hand-feeds every single word through one FIFO register, with memory barriers around each access.
No DRAM, tiny chunks	TrustZone blocks DRAM writes from FEL's non-secure context, so the whole driver lives in 40 KB of SRAM. Writes are capped at 64 sectors (32 KB), and the boot ROM's USB handler times out after ~3 seconds — so 8 MB becomes 256 separate round trips.

If you will be doing repeated eMMC work, you can rebuild the driver for a wider bus. The 1-bit default is the safest and is what this guide was verified with:

```
make clean
make EMMC_DEFS='-DSPEED_8BIT_FAST' fel-emmc.bin # ~8x faster, less proven
```

Verify the write landed

Still in FEL, read the start of the partition back and compare it byte-for-byte against what you meant to write. Two minutes here is far cheaper than a boot loop:

```
./emmc-tool.sh read 599040 256 /tmp/readback.bin
head -c 131072 ~/magisk_patched-30700_XXXXX.img > /tmp/expected.bin
cmp /tmp/readback.bin /tmp/expected.bin && echo "FLASH VERIFIED"
```

WATCH OUT

Do not flash a disabled vbmeta. On most Android devices `vbmeta --flags 0x02` is the standard trick for booting modified images. On this Allwinner bootloader it causes an immediate boot halt — “*your device is corrupt*”. Leave `vbmeta` exactly as it is; the unlocked bootloader already does warn-and-continue.

STEP 6

Boot and confirm root

~3 MIN

```
xfel reset
adb wait-for-device
adb shell su -c id
# uid=0(root) gid=0(root) groups=0(root) context=u:r:magisk:s0

adb shell su -c "magisk -v"
# 30.7:MAGISK:R
```

The first su request raises a prompt on the tablet — grant it. If su reports *"inaccessible or not found"* but `ps -A | grep magiskd` shows the daemon running, the Magisk app simply has not finished first-run setup: open it on the tablet and try again.

NOTE

The first boot after flashing can take noticeably longer than usual. Give it a few minutes before concluding anything has gone wrong.

STEP 7

Remove the bloatware

REVERSIBLE

Use `pm uninstall --user 0` rather than deleting APKs from `/system`. It survives reboots, needs no root, keeps OTA verification intact, and any package can be restored instantly because the APK is never actually removed.

```
adb shell pm uninstall --user 0 <package>      # remove
adb shell cmd package install-existing <package>  # put it back
```

DESTRUCTIVE

Wait about 60 seconds after your last uninstall before rebooting. PackageManager updates its in-memory state immediately but flushes `/data/system/packages.xml` to disk asynchronously. Reboot too quickly and every removal silently reverts — the packages come back and it looks like the ROM restored them. Confirm the flush landed before rebooting:

```
adb shell su -c "stat -c %Y /data/system/packages.xml"  # compare against:
adb shell "date +%s"                                   # ...this
```

WATCH OUT

Remove in **small batches and reboot between them**. If a batch breaks the boot, you know exactly which one to blame. Removing 70 packages at once and then rebooting tells you only that something, somewhere, was load-bearing.

Group 1 — vendor and factory-test junk (9 packages)

All Allwinner/Teclast. None are user-facing and none have a launcher icon. The first is the notable one: KeepTesting is factory burn-in test software left in the shipping image, flagged PERSISTENT so Android keeps it running permanently, holding INTERNET, REBOOT, SHUTDOWN, DISABLE_KEYGUARD and MANAGE_EXTERNAL_STORAGE.

<code>com.clock.ptl.keeptesting</code>	<code>com.softwinner.awlogsettings</code>
<code>com.softwinner.dragonatt</code>	<code>com.softwinner.awsysteminfo</code>
<code>com.softwinner.timerswitch</code>	<code>com.sktl.sktldeviceinfo</code>
<code>com.teclast.update</code>	<code>com.softwinner.qrscanner</code>
<code>com.softwinner.update</code>	

KeepTesting is PERSISTENT, so uninstalling does not kill the already-running process. Either reboot, or kill it directly — it will not come back:

```
adb shell ps -A | grep keeptesting      # note the PID
adb shell su -c "kill -9 <PID>"
```

Group 2 — Google apps and unused services (32 packages)

Reboot and confirm Group 1 stuck before starting this one. Everything here is safe on a device where you do not use these apps; the accessibility entries are only safe if you have no accessibility services enabled — check with `adb shell settings get secure enabled_accessibility_services`.

<code>com.google.android.youtube</code>	<code>com.android.soundrecorder</code>
<code>com.google.android.apps.youtube.music</code>	<code>com.android.calculator2</code>
<code>com.google.android.videos</code>	<code>com.google.android.apps.wellbeing</code>
<code>com.google.android.play.games</code>	<code>com.google.android.apps.restore</code>
<code>com.google.android.apps.books</code>	<code>com.google.android.marvin.talkback</code>
<code>com.google.android.apps.maps</code>	<code>com.google.android.accessibility.switchaccess</code>
<code>com.google.android.gm</code>	<code>com.google.android.apps.accessibility.voiceaccess</code>
<code>com.google.android.apps.docs</code>	<code>com.google.android.apps.carrier.carrierwifi</code>
<code>com.google.android.keep</code>	<code>com.google.android.printservice.recommendation</code>
<code>com.google.android.calendar</code>	<code>com.google.android.tag</code>
<code>com.google.android.apps.tachyon</code>	<code>com.google.android.feedback</code>
<code>com.google.android.apps.photos</code>	<code>com.google.android.onetimeinitializer</code>
<code>com.google.android.apps.nbu.files</code>	<code>com.google.android.googlequicksearchbox</code>
<code>com.google.android.apps.adm</code>	<code>com.softwinner.miracastReceiver</code>
<code>com.google.android.apps.safetyhub</code>	<code>com.google.android.partnersetup</code>
<code>com.google.android.apps.googleassistant</code>	<code>com.google.android.contacts</code>

Side effects worth knowing before you run it:

- `googlequicksearchbox` — removes the launcher search bar and Assistant hotword.
- `partnersetup` — this ROM uses it to set Chrome's homepage to <http://www.teclast.com/>. Removing it may leave Chrome with a blank or broken homepage, which is easily reset in Chrome's own settings.
- `apps.photos` — leaves you with no gallery app unless you install one.

Do not remove these

Every one of these is Google-signed and looks like bloat. All of them are load-bearing.

Package	What breaks
com.google.android.inputmethod.latin	Your only keyboard. The other listed “IME” is Google TTS voice input, not a typing keyboard. Remove this and you cannot type anything.
com.google.android.webview	Any app that renders web content
com.google.android.permissioncontroller	All permission dialogs
com.google.android.packageinstaller	Installing APKs — including sideloading
com.google.android.providers.media.module	Media and file storage access
com.google.android.documentsui	The file picker, used by every app
com.google.android.networkstack (+ .tethering, .config)	Networking
com.google.android.connectivity.resources, wifi.resources	WiFi and connectivity
com.google.android.ext.services, modulemetadata	Framework extensions
com.softwinner.camera2	The camera app
com.softwinner.awmanager	Allwinner's background management service (“awbms”). The framework calls into it by name, and it is what restrains vendor apps from running freely.
com.softwinner.dramdfs-service	Not a package at all — a native daemon doing DRAM frequency scaling. Leave it.

DESTRUCTIVE

Removing GMS will probably boot-loop this device. Taking out com.google.android.gms, com.google.android.gsf and the mainline modules (com.google.mainline.*, sdksandbox, adservices.api, ondevicepersonalization.services, federatedcompute, cellbroadcastservice, captiveportallogin) produced an unbootable device here that needed a factory reset. The com.google.mainline.* packages in particular are module metadata the framework verifies at boot — they are not bloat. If you want a fully de-Googled tablet, a custom ROM or a GSI is the right tool, not pm uninstall.

Result

Packages	199 → 158 (41 removed)
Memory available	~2,165 MB → ~2,446 MB
Persistent factory-test process	Gone entirely
Survives reboot	Yes — verified

Recovery

Boot loop after removing packages

A factory reset fixes this completely, and costs less than it sounds. Package removals live in `/data`, so a reset restores **all** of them — the APKs were never deleted. Boot to recovery (Volume Up + Power) and choose *Wipe data / factory reset*.

NOTE

Root survives a factory reset. Magisk lives in the patched `init_boot` partition, which a data wipe does not touch. Afterwards `magiskd` is already running; you only need to reinstall the Magisk *app* and re-grant shell root.

Boot loop after flashing, or no boot at all

Restore the backup through FEL. This works even if Android never starts, because FEL is in mask ROM:

```
# Enter FEL with the button sequence (Step 2)
cd A523-root/tools
./emmc-tool.sh gpt
./emmc-tool.sh flash init_boot_a ~/p30-backups/init_boot_a_original.img
xfel reset
```

Removing root entirely

Flash the original `init_boot_a_original.img` back exactly as above. To also re-lock the bootloader, `fastboot oem lock` — but note that re-locking a device with modified partitions can leave it unbootable, so restore the stock image first and confirm it boots.

If your bootloader will not unlock

If `ro.boot.flash.locked` stays at 1 after `fastboot oem unlock`, your unit stores the flag only in Secure Storage and it must be written directly. The A523-root repository ships a tool for this. Back up Secure Storage first — it also holds your MAC addresses and serial:

```
./emmc-tool.sh read 12288 256 ss.bin
python3 unlock-bootloader.py patch ss.bin ss_unlocked.bin
python3 unlock-bootloader.py verify ss_unlocked.bin
./emmc-tool.sh write 12288 ss_unlocked.bin
```

Things that will waste your time

Each of these cost real time during the run this guide is based on.

Symptom	Cause and fix
sunxi-fel aborts with an AWUS assertion	Wrong tool. The packaged sunxi-tools does not speak the A523's FEL dialect. Build xfel from git instead.
FEL device appears then vanishes instantly	You used <code>adb reboot fel</code> . It only blips. Use the button sequence, which holds indefinitely.
<code>fastboot getvar all</code> → “secure mode, fastboot limited used”	Normal. This bootloader implements a tiny command subset. Nothing is wrong; use FEL.
<code>fastboot flashing unlock</code> → “unknown cmd”	Use the older spelling: <code>fastboot oem unlock</code> .
Flashing 8 MB takes ten minutes	Expected — 1-bit bus, PIO not DMA, 32 KB chunks, no DRAM. See Step 5. Do not interrupt it.
Uninstalled packages return after a reboot	You rebooted before PackageManager flushed <code>packages.xml</code> . Wait ~60 s and verify the file's mtime first.
Device boot-loops after a big debloat	Almost certainly GMS/GSF or a <code>com.google.mainline.*</code> module. Factory reset restores every package; root survives.
“Connected, no internet” but internet works	This ROM's captive-portal check targets <code>www.google.cn</code> , which is unreachable from most networks (and is on many DNS blocklists). Cosmetic only — see below.
adb cannot see the device after unlocking	The wipe cleared Developer options and the ADB key. Re-enable USB debugging and accept the RSA prompt again.
Magisk installed but <code>su</code> : not found	Open the Magisk app once to finish first-run setup, then retry.

The “no internet” warning

Android decides whether a network works by fetching a `/generate_204` URL. Chinese-market ROMs point that at `www.google.cn`. If that host does not resolve on your network, the check fails forever and you get a permanent warning on a perfectly working connection. Point it somewhere reachable:

```
adb shell settings put global captive_portal_http_url \
    http://connectivitycheck.gstatic.com/generate_204
adb shell settings put global captive_portal_https_url \
    https://connectivitycheck.gstatic.com/generate_204

# or silence the check completely:
adb shell settings put global captive_portal_mode 0

# revert to the ROM default:
adb shell settings delete global captive_portal_http_url
```

Confirm the network genuinely works first, so you are fixing a cosmetic bug rather than hiding a real one: `adb shell ping -c 2 8.8.8.8`.

A note on what this tablet actually phones home

A DNS capture across a first connection and several minutes of idle showed only Google endpoints plus `pool.ntp.org`. No Allwinner, Teclast, or Chinese analytics traffic originated from the firmware — including from KeepTesting, which holds the `INTERNET` permission but never used it. One `hm.baidu.com` (Baidu Tongji) lookup did appear, but it came from loading Teclast's website — which is Chrome's OEM-set homepage — not from a background service. Worth repeating on your own unit if it matters to you; this was a short window, and it rules out aggressive beaconing rather than a daily check-in.

Built and verified on a Teclast P30 (Allwinner A523, Android 14) in a single session. Tooling by the [linux-sunxi](#) community, [xboot/xfel](#), [chrislennon/A523-root](#), and [topjohnwu/Magisk](#).